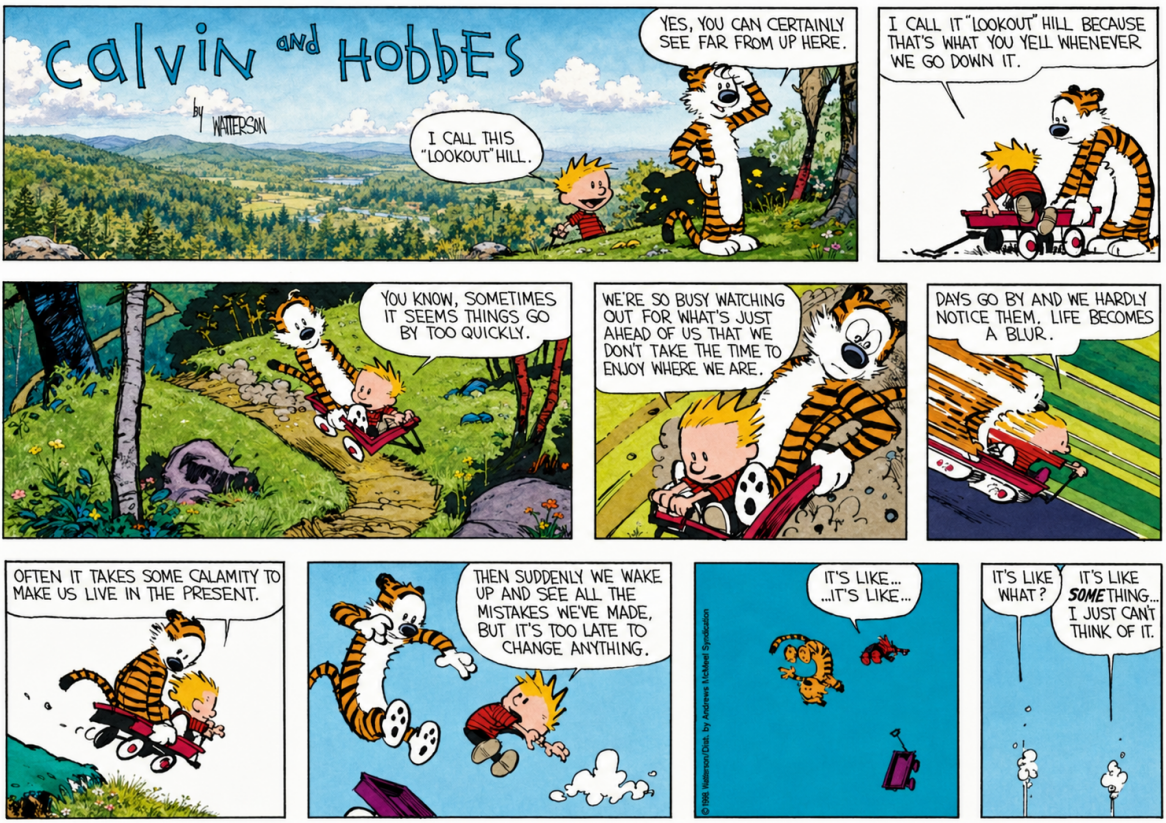
Embeddings Detail and RAG

We will begin 8:35 PM



What is RAG and Why do we need it?

GPT- 5- nano

→ Close to 380K window → Switch to Larger model
→ Compact (Lossy compression)
→ Remove top 50K tokens

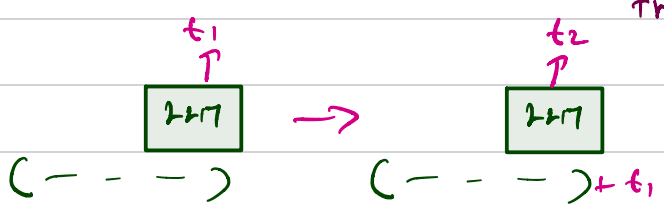
S_1 → 3L → r_1
S_2 → 60K → r_2

Quality (R_1) < Quality (R_2)

Context rot

Customer Service Chatbot

50k pdf files → query = " How do I print on transparent pixels "

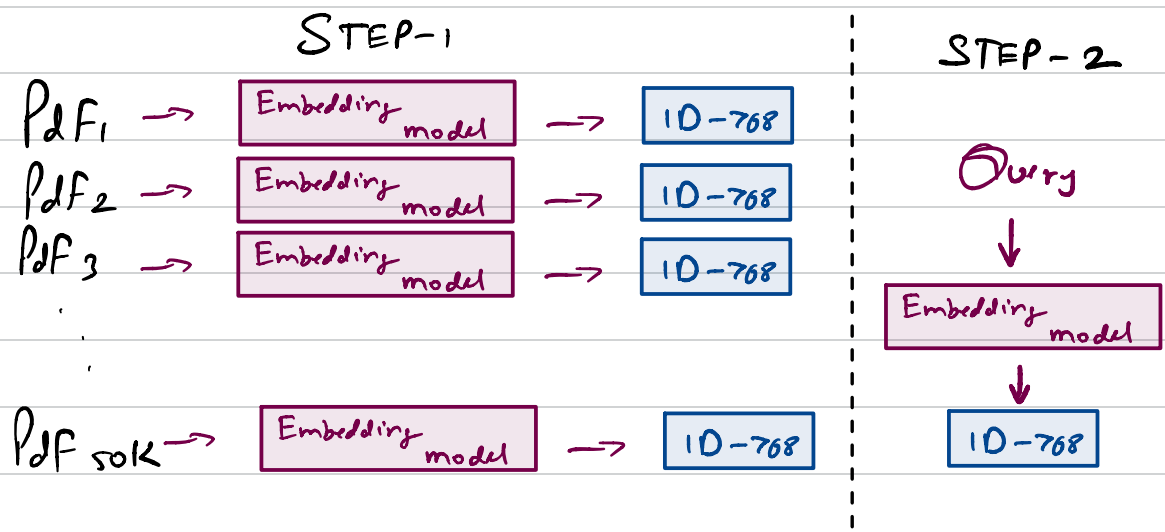


Embedding Model

Query → GPT → tokens / words

Query → Embeddin → Embedding.

RAG → Retrival Augmented Generation.



STEP- 3

Sort on Cos. sim score

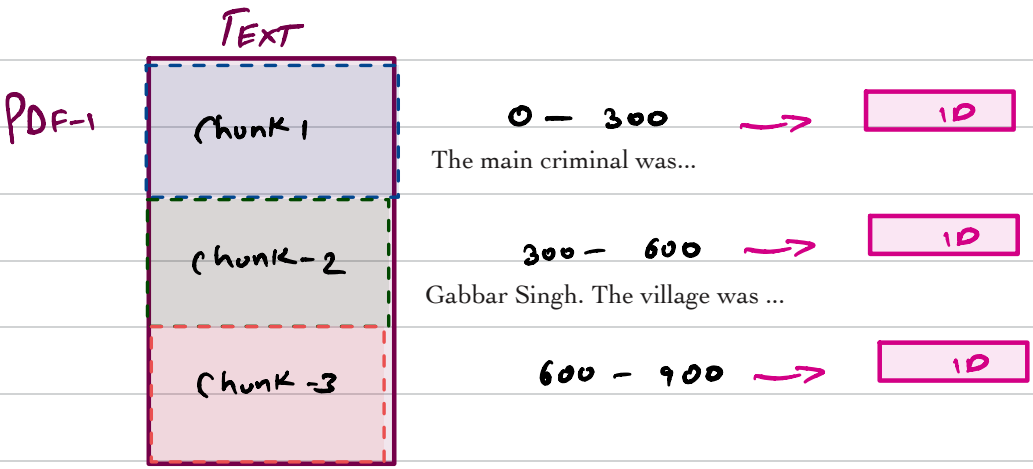
and sort → top 5 scores.

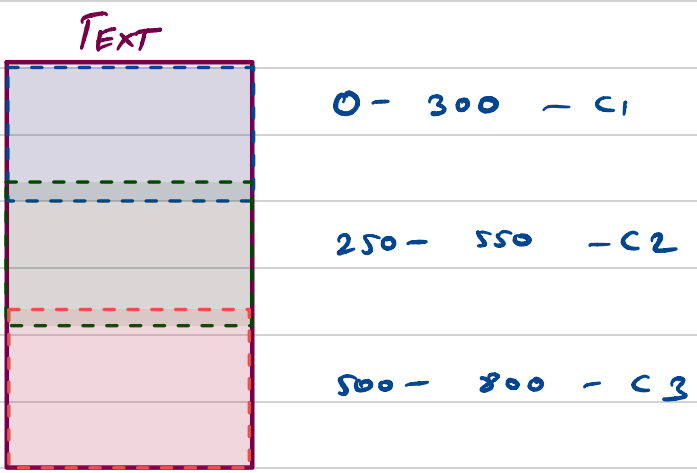
Cosine-Sim (E-P1, Query)

Cosine-Sim (E-P2, Query)

Cosine-Sim (E-P50K Query)

Chunk your documents





window size = 2

Chunk 6, Chunk 8 and Chunk 23 are most relevant.

Chunk 4, 5, 6, 7, 8, 9, 10
Chunk 21, 22, 23, 24, 25
) – LLM + Query

How to compute cosine similarity for millions of chunks?

PdF – 50K → 20 chunks

$5 \times 10^4 \times 20$

10^6 – 1 million chunks

We've 1 Billion chunks – 10^9

↓

$10^9 \times 768$

Vanilla computation: 10^9

Query Vector → Embedding. – 768

K_1 – 100M — SK_1 – 10M, SK_2 – 10M, ⋱ SK_{10} – 10M

V_1 – Computations = 10 + 100M

K_2 → 100M — SK_1 – 10M, SK_2 – 10M, SK_{10} – 10M

V_2 – Computations : 10 + 10 + 10M

Approx : $\frac{10^9}{10 \times 10^6}$

K_{10} – 100M — SK_1 – 10M, SK_2 – 10M, ⋱ SK_{10} – 10M

EmbeddingGemma

1. Just the query, embed(query) - embedding
2. have prefix of "Title: "Name of document", "content:" content
3. you should have prefix of "task: search_text" and then query"